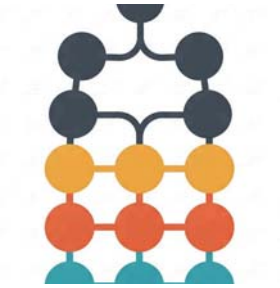
# B 3.2.2 Polymorphism (HL Only)



B3.2.2 Construct code to model polymorphism and its various forms, such as method overriding. (HL Only)

This section explores polymorphism in OOP, emphasising its role in code flexibility and reusability. It covers implementing dynamic polymorphism (e.g., method overriding) and applying static polymorphism to optimise code efficiency.

## Learning Objectives Java

- Students will be able to explain the principle of polymorphism and its contribution to code flexibility and reusability.
- Students will be able to implement dynamic polymorphic behaviour in Java through method overriding.
- Students will be able to apply static polymorphic behaviour in Java through method overloading to maximise code efficiency.
- Students will be able to identify scenarios where polymorphism can be effectively applied in object-oriented programming.

**Relevant ATL Skills:**

- **Thinking Skills:**
  - **Abstraction:** Students will abstract common behaviours into superclasses using interfaces or abstract classes.
  - **Transfer:** Students will apply the concept of polymorphism to different scenarios.
  - **Critical thinking:** Students will analyse the benefits and drawbacks of using polymorphism and overloading.
  - **Problem-solving:** Students will use polymorphism and overloading to design more flexible and reusable code.
- **Communication Skills:**
  - Students will use appropriate terminology (e.g., override, overload, upcasting) to explain polymorphism.
  - Students will discuss and compare different implementations of polymorphic behaviour.
- **Self-Management Skills:**

    - **Organisation:** Students will structure their Java code using inheritance, method overriding, and overloading effectively.

## Learning Objectives Python

- Students will be able to explain the principle of polymorphism and its contribution to code flexibility and reusability.
- Students will be able to implement dynamic polymorphic behaviour in Python through method overriding.
- Students will be able to explain how static polymorphic behaviour can maximise code efficiency in Python (method overloading concepts, acknowledging Python's implicit handling).
- Students will be able to identify scenarios where polymorphism can be effectively applied in object-oriented programming.

**Relevant ATL Skills:**

- **Thinking Skills:**
    - **Abstraction:** Students will abstract common behaviours into superclasses.
    - **Transfer:** Students will apply the concept of polymorphism to different scenarios.
    - **Critical thinking:** Students will analyse the benefits and drawbacks of using polymorphism.
    - **Problem-solving:** Students will use polymorphism to design more flexible and reusable code.
- **Communication Skills:**
    - Students will use appropriate terminology to explain polymorphism and its implementation.
    - Students will discuss and compare different implementations of polymorphic behaviour.
- **Self-Management Skills:**
    - **Organisation:** Students will structure their code using inheritance and method overriding effectively.

# Lesson Resources

## Presentation Java

**Link to presentation**

## Presentation Python

**Link to presentation**

## eBook

**B 3.2.2 Polymorphism Java (HL Only)**

**B 3.2.2 Polymorphism Python (HL Only)**

## Other Resources

**B 3.2.2 Worksheet Java**

**B 3.2.2 Worksheet Java Answers**

**B 3.2.2 Worksheet Python**

**B 3.2.2 Worksheet Python Answers**

# Lesson Planning

## Lesson Plan Java

| Time (minutes) | Activity | Content Covered | Slide(s) |
|---|---|---|---|
| 0-5 | **Introduction & Review** | Recap of classes, objects, and inheritance in Java. Introduction to the concept of "many forms" (polymorphism). | 1-2 |
| 5-15 | **Explaining Polymorphism** | Defining polymorphism, illustrating its benefits (flexibility, reusability). Real-world analogies. | 3-5 |
| 15-25 | **Dynamic Polymorphism: Method Overriding** | Explanation of method overriding in Java, the `@Override` annotation, the concept of upcasting. | 6-8 |
| 25-40 | **Java Example & Demonstration** | Live coding demonstration in Java showcasing method overriding with a clear example (e.g., different shapes with an `calculateArea()` method). | 9-12 |
| 40-50 | **Static Polymorphism: Method Overloading** | Explanation of method overloading in Java, defining methods with the same name but different parameter lists. | 13-14 |
| 50-55 | **Activity: Code Analysis & Prediction** | Students analyse short Java code snippets demonstrating polymorphism and predict the output (worksheet). | 15 |
| 55-60 | **Wrap-up & Q&A** | Review of key concepts, answering student questions, linking back to the learning objectives. | 16 |

## Lesson Plan Python

| Time (minutes) | Activity | Content Covered | Slide(s) |
|---|---|---|---|
| 0-5 | **Introduction & Review** | Recap of classes, objects, and inheritance. Introduction to the concept of "many forms" (polymorphism). | 1-2 |
| 5-15 | **Explaining Polymorphism** | Defining polymorphism, illustrating its benefits (flexibility, reusability). Real-world analogies. | 3-5 |
| 15-25 | **Dynamic Polymorphism: Method Overriding** | Explanation of method overriding, how subclasses can provide specific implementations of inherited methods. Python syntax for overriding. | 6-8 |
| 25-40 | **Python Example & Demonstration** | Live coding demonstration in Python showcasing method overriding with a clear example (e.g., different shapes with an `area()` method). | 9-12 |
| 40-50 | **Static Polymorphism (Conceptual)** | Discussion of static polymorphism (method overloading) – explaining the concept but noting Python's dynamic typing and how it's implicitly handled through default arguments or variable arguments. | 13-14 |
| 50-55 | **Activity: Code Analysis & Prediction** | Students analyse short Python code snippets demonstrating polymorphism and predict the output (worksheet). | 15 |
| 55-60 | **Wrap-up & Q&A** | Review of key concepts, answering student questions, linking back to the learning objectives. | 16 |

## Teacher Notes (Addressing Challenges) Java:

- **Distinguishing Overriding and Overloading:** Students often confuse these two concepts. Emphasise that overriding happens in different classes (superclass and subclass) with the same method signature, while overloading happens within the same class with the same method name but different parameter lists. Use clear examples to illustrate the difference.
- **The `@Override` Annotation:** Stress the importance of using the `@Override` annotation in Java. Explain that it's not strictly necessary for the code to compile, but it is crucial for catching errors and improving code readability.

- **Upcasting and Method Resolution:** Ensure students understand that even when an object is upcasted to its superclass type, the subclass's implementation will be executed if an overridden method is called.
- **Rules of Method Overloading:** Explain the rules for method overloading in Java (same name, different parameter lists). Briefly mention that return type alone is not sufficient for overloading.
- **Real-world Java Examples:** Provide real-world examples from common Java libraries (e.g., the `System.out.println()` method is overloaded) to show the practical applications of overloading.
- **Abstract Classes and Interfaces:** For HL students, briefly discuss how abstract classes and interfaces are often used to define polymorphic interfaces in Java, enforcing that subclasses implement certain methods.
- **Debugging Polymorphic Java Code:** Guide students through using a debugger to step through polymorphic code and observe which methods are being executed at runtime.

## Teacher Notes (Addressing Challenges) Python:

- **Confusion between Inheritance and Polymorphism:** Students might confuse the "is-a" relationship of inheritance with the "can also act as" aspect of polymorphism. Emphasise that inheritance sets up the hierarchy, while polymorphism allows objects within that hierarchy to be treated in a general way. Use analogies carefully.
- **Understanding Runtime Binding:** The concept that the method executed is determined at runtime can be tricky. Walk through the code execution step by step during the live coding demonstration, highlighting how Python decides which `area()` or `speak()` method to call based on the object's actual type.
- **Static Polymorphism in Python:** Be clear that Python doesn't have traditional method overloading like Java due to its dynamic typing. Focus on the *concept* of static polymorphism (choosing the method at compile time based on arguments) and how Python achieves similar flexibility with default and variable arguments. Avoid getting bogged down in a detailed comparison of language features.
- **Real-world Examples:** Continuously refer back to real-world examples and analogies to make the abstract concepts more concrete and relatable. Encourage students to think of their own examples.
- **Debugging Polymorphic Code:** Students might encounter issues when writing their own polymorphic code. Guide them in checking the class hierarchy, ensuring method names and parameters match for overriding, and using print statements to trace the execution flow and identify which methods are being called.
- **The `super()` Keyword:** Briefly explain the use of `super()` when overriding methods, demonstrating how it allows a subclass to call the superclass's version of the method. This can help extend functionality.

## Report a problem